

**Practice Exercise 1**

Write a C++ program to calculate the density using class and object.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
    // data members (variables)  
    // member functions (methods)  
};  
  
int main() {  
    ClassName obj; // creating an object  
}
```

Output:

Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Declare a Car Class with data members and member functions using private and public access specifiers.

**Practice Exercise 2**

Write a C++ program to calculate the density showcasing friend class.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class B; // Forward declaration if needed
```

```
class A {  
    friend class B; // B is a friend of A  
private:  
    int data;  
};
```

Output:

Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Implement a Temperature class with a friend function to convert temperature from Celsius to Fahrenheit.

**Practice Exercise 3**

Write a C++ program to calculate the density showcasing friend function.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
private:  
    // private data members  
  
public:  
    friend returnType functionName(ClassName); // Friend function declaration  
};
```

Output:

Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Implement a Temperature class with a friend function to convert temperature from Celsius to Fahrenheit.

**Practice Exercise 4**

Write a C++ program to calculate the density using a default constructor.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
public:  
    ClassName() {  
        // Initialization code (if any)  
    }  
};
```

Output:

Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Implement a Car class with a default constructor to initialize brand and model, and a function to display the car details.

**Practice Exercise 5**

Write a C++ program to calculate the density using a parameterized constructor.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
public:  
    ClassName(type1 arg1, type2 arg2, ...) {  
        // Initialize data members using arguments  
    }  
};
```

Output:

Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Implement a Car class with a parameterized constructor to initialize brand and model, and a function to display the car details.

**Practice Exercise 6**

Write a C++ program to calculate the density using a copy constructor.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
public:  
    ClassName(const ClassName &obj) {  
        // Copy data from obj  
    }  
};
```

Output:

Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Implement a Car class with a copy constructor to initialize brand and model details from another object and a function to display the car details.

**Practice Exercise 7**

Write a C++ program to calculate the weight density of water demonstrating encapsulation.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
private:  
    // private data members  
  
public:  
    // public methods to access/modify private data  
};
```

Output:

Weight Density of Fluid: <computed value> N/m³

Task Evaluation Exercise:

Implement a Solid class with member functions to set mass and volume, and calculate the weight density.

**Practice Exercise 8**

Write a C++ program to demonstrate single inheritance by calculating the density and relative density of a fluid.

Software:

C++, g++ compiler, VS Code

Syntax:

```
// Base class
class A {
    // members of A
};

// Derived class from A
class B : public A {
    // members of B
};
```

Output:

Relative Density of Fluid: <computed value>

Task Evaluation Exercise:

Implement a Material class using single inheritance to calculate mass density and relative density.

**Practice Exercise 9**

Write a C++ program to demonstrate multilevel inheritance by calculating the density, relative density, and weight of a fluid.

Software:

C++, g++ compiler, VS Code

Syntax:

```
// Base class
class A {
    // members of A
};

// Derived class from A
class B : public A {
    // members of B
};

// Derived class from B
class C : public B {
    // members of C
};
```

Output:

Weight of Fluid: <computed value> N

Task Evaluation Exercise:

Implement a Gas class using multilevel inheritance to calculate the density, relative density, and weight of a gas under specific conditions.

**Practice Exercise 10**

Write a C++ program to demonstrate multiple inheritance by calculating the density, relative density, and weight of a fluid.

Software:

C++, g++ compiler, VS Code

Syntax:

```
// Base classes
class A {
    // members of A
};

class B {
    // members of B
};

// Derived class inherits from both A and B
class C : public A, public B {
    // members of C
};
```

Output:

Weight of Fluid: <computed value> N

Task Evaluation Exercise:

Implement a Gas class using multiple inheritance to calculate the density, relative density, and weight of a gas under specific conditions.

**Practice Exercise 11**

Write a C++ program to demonstrate hierarchical inheritance by calculating the density, relative density, and weight of a fluid.

Software:

C++, g++ compiler, VS Code

Syntax:

```
// Base class
class A {
    // members of A
};

// Multiple derived classes from the same base
class B : public A {
    // members of B
};

class C : public A {
    // members of C
};
```

Output:

Relative Density: [computed value]

Weight of Fluid: [computed value] N

Task Evaluation Exercise:

Implement a Gas class using hierarchical inheritance to calculate the density, relative density, and weight of a gas under specific conditions.

**Practice Exercise 12**

Write a C++ program to demonstrate hybrid inheritance by calculating the density, relative density, and weight of a fluid.

Software:

C++, g++ compiler, VS Code

Syntax:

```
// Base class
class A {
    // members of A
};
// Derived class from A
class B : public A {
    // members of B
};
// Another derived class from A
class C : public A {
    // members of C
};
// Hybrid derived class from B and C
class D : public B, public C {
    // members of D
};
```

Output:

Density: <density> kg/m³

Relative Density: <relative_density>

Weight of Fluid: <weight> N

Task Evaluation Exercise:

Implement a Gas class using hybrid inheritance to calculate the density, relative density, and weight of a gas under specific conditions.

**Practice Exercise 13**

Write a C++ program to demonstrate operator overloading by implementing vector addition.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
public:  
    returnType operator+ (const ClassName &obj) {  
        // Operator overloading logic  
    }  
};
```

Output:

Vector 1: (x1, y1, z1)

Vector 2: (x2, y2, z2)

Vector Sum: (x1 + x2, y1 + y2, z1 + z2)

Task Evaluation Exercise:

Implement a Vector class with operator overloading for vector subtraction.

**Practice Exercise 14**

Write a C++ program to demonstrate function overloading by calculating areas of different shapes.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class ClassName {  
public:  
    void functionName();           // No parameters  
    void functionName(int);        // One int parameter  
    void functionName(string, int); // Two parameters of different types  
};
```

Output:

Area of Square: <value>

Area of Rectangle: <value>

Task Evaluation Exercise:

Implement a Vector class with operator overloading for vector subtraction.

**Practice Exercise 15**

Write a C++ program to demonstrate abstract classes and virtual functions by calculating the density, relative density, and weight of a fluid.

Software:

C++, g++ compiler, VS Code

Syntax:

```
class Base {  
public:  
    virtual void functionName() = 0; // Pure virtual function → makes class abstract  
};
```

Output:

Relative Density: <computed value>

Weight: <computed value> N

Task Evaluation Exercise:

Implement an abstract class, Shape, with a pure virtual function area(), and derive Triangle and Square classes to calculate their respective areas.

**Practice Exercise 16**

Write a C++ program to demonstrate file handling operations - open, close, and update using density calculation.

Software: C++, g++ compiler, VS Code

Syntax:

```
ofstream outFile("filename.txt"); // Open file for writing
outFile << "Some data...";        // Write to file
outFile.close();                  // Close the file
```

```
ifstream inFile("filename.txt"); // Open file for reading
string line;
while (getline(inFile, line)) {
    cout << line << endl;        // Read and display line by line
}
inFile.close();                  // Close the file
```

```
fstream file("filename.txt", ios::app); // Open file in append mode
file << "Appended data...";            // Add new data
file.close();                          // Close the file
```

Output:

Create, read, and Append a file named output.txt to have following data:

Mass: <value>

Volume: <value>

Density: <value>

Updated: output calculated successfully.

Task Evaluation Exercise:

Write a program to calculate and store the relative density of a substance by comparing it to a reference density value and append the results to the file.

**Practice Exercise 17**

Write a C++ program to demonstrate the use of Command Line Arguments for calculating stress.

Software: C++, g++ compiler, VS Code

Syntax:

```
int main(int argc, char* argv[]) {  
    // argc = argument count  
    // argv[] = argument vector (array of C-strings)  
}
```

Output:

```
$ ./programName 500 25  
Stress: 20 N/m^2
```

Task Evaluation Exercise:

Write a program to accept force, area, strain from command-line and calculate the modulus of elasticity.

**Practice Exercise 18**

Write a C++ program to demonstrate the use of Function Templates for calculating stress.

Software: C++, g++ compiler, VS Code

Syntax:

```
template <typename T>
T functionName(T parameter1, T parameter2) {
    // function body using parameters of type T
}
```

Output:

```
Enter force (integer, N): <value>
Enter area (integer, m^2): <value>
Integer Stress: <value> N/m^2
Enter force (float, N): <value>
Enter area (float, m^2): <value>
Float Stress: <value> N/m^2
```

Task Evaluation Exercise:

Write a program using a function template to accept force and area (as command-line inputs) and calculate pressure.

**Practice Exercise 19**

Write a C++ program to demonstrate the use of Exception Handling for calculating stress.

Software: C++, g++ compiler, VS Code

Syntax:

```
try {  
    // Code that might throw an exception  
    if (some_error_condition) {  
        throw exception_type("Error message");    }  
}  
catch (exception_type& e) {  
    // Code to handle the exception  
    cout << "Exception caught: " << e.what() << endl;  
}
```

Output:

Without Exception:

Enter force: 20
Enter area: 5
Stress: 4 N/m²

With Exception:

Enter force: 20
Enter area: 0
Exception caught: Error: Area cannot be zero (division by zero is undefined).

Task Evaluation Exercise:

Write a program to accept force, area, strain from command-line and calculate the modulus of elasticity.